

ECE 457A Midterm Cheatsheet

Search & Metaheuristics. Covers: foundations, uninformed/informed search, minimax, HC/SA/Tabu, GA.

1 Foundations & Problem Solving

Well- vs Ill-Structured

Well-structured: state space, initial state, operators, goal test, cost all explicit; problem tuple

$P = (S, s_0, A, G, c)$.

Solution: path $s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_g$ with $G(s_g) = 1$.

Ill-structured: goals shift, constraints incomplete/hidden, path unclear; needs framing + representation first. Hadamard well-posed: solution (i) exists, (ii) unique, (iii) stable (depends continuously on data).

Optimization as Search

$\min_{x \in X} f(x)$
s.t. $g_i(x) \leq 0, h_j(x) = 0$.

Feasible set

$F = \{x \in X \mid g_i(x) \leq 0, h_j(x) = 0\}$.

Search = iterative loop: evaluate $f(\cdot)$, pick next via neighbourhood $N(\cdot)$, stop on budget/target.

Exact: optimal but costly.

Approximate/heuristic: near-optimal, cheap, no optimality guarantee.

Complexity (engineering)

P: exact poly-time $O(n^k)$ (sort, shortest path, MST).

NP: solution verifiable in poly-time (SAT, subset-sum).

NP-hard: $\geq$ hardest NP, no known poly algorithm (TSP, scheduling, placement).

NP-complete: in NP and NP-hard.

Blow-up: binary choices $|S| = 2^n$; TSP tours $|S| = (n-1)!/2$; tree search $O(b^d)$.

Verify-easy $\neq$ discover-easy: trade solution quality $\leftrightarrow$ time/memory/evaluations.

Constructive vs Local

Constructive: build from empty partial state, add components by a rule (greedy).

Local search: start from a complete state, apply move operator, keep if evaluation improves.

Neighbourhood $N(s) = \{\text{states reachable from } s \text{ by one move}\}$; defines landscape + "improvement".

Goal then Problem Formulation

Goal formulation (first): decide what matters / what to ignore; defines termination.

Problem formulation: specify $\langle S, s_0, A, T, G, c \rangle$ + representation. Many "algorithm failures" are really formulation failures (bad encoding, weak/strong actions, misaligned cost).

Constraint Handling

Reject illegal moves; Repair/project to feasible; Penalty $f'(x) = f(x) + \lambda v(x)$ (v =violation). Be consistent: mixed policies destabilise search.

Agent / Environment Dimensions

Observable vs partially observable; deterministic vs stochastic; static vs dynamic; discrete vs continuous; single- vs multi-agent; episodic vs sequential.

Closed-World Assumption: state description complete + trustworthy (full obs., deterministic, sequential, static).

Goal-based: stop at first $G(s) = 1$.

Utility-based: maximise $U(s) \in \mathbb{R}$.

Performance Measures

Completeness: finds a solution if one exists.

Optimality: finds the least-cost solution.

Time / Space complexity in: b =branching factor, d =depth of shallowest goal, m =max path length. State-space size proxy $|V| + |E|$.

Example State-Space Sizes

8-puzzle: $|S| = 9!/2 = 181440$

(parity-reachable half).

Grid $m \times n$, k blocked: $|S| = mn - k$.

Symmetric TSP: $|S| = (n-1)!/2$.

Missionaries/cannibals: state (M_L, C_L, B) , constraint per bank $M=0$ or $M \geq C$.

2 Uninformed (Blind) Search

Generic Search Loop

Build search tree over problem graph; repeat: choose a node from fringe, goal-test, expand (generate children). Strategy = how the next fringe node is chosen (the data structure). Node states: explored / on frontier (fringe) / unexplored. Repeated states: keep lowest-cost; check on generation (adding to fringe) or on expansion (popping).

BFS

Expand shallowest node; fringe = FIFO queue; new successors to the back.

Nodes generated to depth d :

$1 + b + b^2 + \dots + b^d = O(b^{d+1})$.

Complete (if b finite); optimal only if path cost = depth (equal step costs); returns shallowest goal.

Time $O(b^{d+1})$, Space $O(b^{d+1})$.

Uniform-Cost Search (UCS)

Expand cheapest node by path cost $g(n)$; fringe = priority queue.

Goal test ON EXPANSION (not generation): a generated goal may be costlier than a queued node, optimal only when it reaches the front. Needs step cost $\geq \epsilon > 0$ (no zero/negative edges). = Dijkstra; = BFS when costs equal.

Complete + optimal. Effective depth $\lfloor C^*/\epsilon \rfloor$.

Time/Space $O(b^{1+\lceil C^*/\epsilon \rceil})$;

C^* =optimal cost.

DFS

Expand deepest node; fringe = LIFO stack.

NOT complete (infinite depth / loops); NOT optimal.

Time $O(b^m)$; Space $O(bm)$ (linear) – main advantage.

Can beat BFS for dense solution spaces.

Depth-Limited Search (DLS)

DFS with cutoff l ; always terminates. Complete iff $d \leq l$ (fails if $l < d$); not optimal.

Time $O(b^l)$, Space $O(bl)$. If $l \gg d$, costlier than needed.

Iterative Deepening (IDS/DFID)

Run DLS with $l = 0, 1, 2, \dots$. Combines DFS space with BFS completeness. Complete (if b finite); optimal if path cost non-decreasing in depth.

Time $O(b^d)$, Space $O(bd)$.

Re-expansion overhead is small (lowest levels dominate).

Bidirectional

Search forward from s_0 and backward from goal; meet in middle.

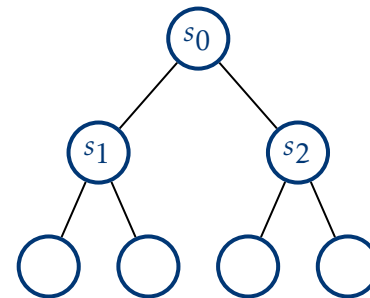
Time/Space $\approx O(b^{d/2})$; needs invertible actions + known goal state.

Comparison

	Compl.	Opt.	Time	Space
BFS	Y^b	Y^c	b^{d+1}	b^{d+1}
UCS	Y	Y	$b^{1+C^*/\epsilon}$	$b^{1+C^*/\epsilon}$
DFS	N	N	b^m	bm
DLS	$Y^{l \geq d}$	N	b^l	bl
IDS	Y^b	Y^c	b^d	bd
Bidir	Y^b	Y^c	$b^{d/2}$	$b^{d/2}$

b : b finite. c : equal/non-decreasing step costs.

Search Tree Anatomy



Root s_0 ; expand = generate children; fringe = generated-but-not-expanded leaves.

3 Informed (Heuristic) Search

Heuristics

$h(n)$ = estimated cost from n to nearest goal; $h(n) \geq 0$, $h(\text{goal}) = 0$.

True cost $h^*(n)$ = minimal cost $n \rightarrow$ goal.

Admissible: never overestimates, $h(n) \leq h^*(n)$.

Consistent (monotone):

$h(n) \leq c(n, n') + h(n')$. Consistent $\Rightarrow$ admissible (not vice versa); then $f = g + h$ never decreases along a path.

Null $h = 0$: admissible, acts like UCS. Perfect $h = h^*$: expands only optimal-path nodes.

Dominance: if

$h_1(n) \leq h_2(n) \leq h^*(n) \forall n$, then h_2 better-informed; A^* with h_2 expands $\subseteq$ nodes of h_1 .

Derive h by relaxation of constraints. 8-puzzle: h_1 = misplaced tiles, h_2 = Manhattan dist (h_2 dominates).

Best-First / Greedy

Best-first: order fringe by evaluation $f(n)$ (priority queue).

Greedy best-first: $f(n) = h(n)$ only. Not complete (loops), not optimal;

Time/Space $O(b^{m+1})$ worst, good h helps a lot.

Beam Search

Informed BFS keeping only best β nodes per level (beam width);

$f(n) = h(n)$.

Not complete, not optimal, not admissible; may discard a solution-path node.

Time $O(\beta d)$, Space $O(\beta d)$.

A* Search

$f(n) = g(n) + h(n)$: path-so-far + estimate-to-go.

Optimal if h admissible (tree search) / consistent (graph search); complete; optimally efficient (no same- h algorithm expands fewer nodes). Limitation: nodes in goal contour still exponential for most problems.

A* Worked Example

Start A , goal J . $f = g + h$:

$f(B) = 6 + 8 = 14$, $f(F) = 3 + 6 = 9 \Rightarrow F$.

$f(G) = (3+1) + 5 = 9$, $f(H) =$

$(3+7) + 3 = 13 \Rightarrow G$.

$f(I) = (3+1+3) + 1 = 8 \Rightarrow I$.

$f(J) = (3+1+3+3) + 0 = 10 \Rightarrow J$.

Path $A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$.

Informed vs Uninformed

Uninformed: only problem definition.

Informed: extra goal-direction info via $h(n)$; closer h to $h^* \Rightarrow$ fewer expansions.

4 Two-Player Game Search

Minimax

MAX (us) maximises, MIN (opponent) minimises; players alternate.

Back up values from terminals: MAX node = max of children; MIN node = min of children.

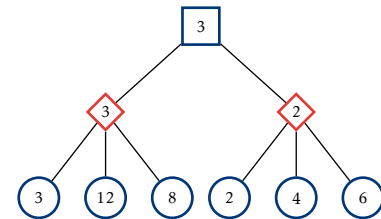
Optimal vs an optimal opponent; minimises the maximum loss the opponent can force.

Real games: cut off at depth, use evaluation $e(p)$ (e.g.

$e = \#open(X) - \#open(O)$); horizon effect.

DFS-based: Time $O(b^m)$, Space $O(m)$. TTT $b \approx 5, d = 9$; checkers $b \approx 35$.

Minimax Tree (root = 3)



MIN picks {3,2}; MAX picks $\max(3,2) = 3$. (box = MAX, diamond = MIN.)

Alpha-Beta Pruning

α = best (max) value MAX can guarantee so far; β = best (min) value MIN can guarantee.

Prune subtree when $\alpha \geq \beta$ (cannot affect the result).

Same value as minimax; move ordering is decisive.

Worst case $O(b^d)$; perfect ordering $O(b^{d/2})$, so effective branching $\approx \sqrt{b} \Rightarrow$ doubles reachable depth for same cost.

Alpha-Beta on Tree Above

Left MIN returns $3 \Rightarrow \alpha = 3$ at the root.

Right MIN examines its first child $2 \Rightarrow \beta = 2$; since $\alpha = 3 \geq \beta = 2$, prune its remaining children (the 4 and 6 are never examined).

Lesson: order best moves first to maximise pruning.

5 Trajectory Metaheuristics

A trajectory method = (i) representation s , (ii) neighbourhood $N(s)$, (iii) acceptance/memory control. The move operator decides which local optima are reachable.

Hill Climbing

Move to best neighbour:

$s_{k+1} \in \arg \min_{s' \in N(s_k)} E(s')$ (descent) until no improvement.

Variants: simple/first-choice (take first improving), steepest-ascent (best neighbour), stochastic, random-restart.

Fails on: local optima, plateaus (flat), ridges. Fixes: restarts, sideways moves, larger N .

NOT complete, NOT optimal; Time $O(\infty)$, Space $O(b)$.

Simulated Annealing

Accept neighbour s' with

$\Delta E = E(s') - E(s)$:

$$P = \begin{cases} 1 & \Delta E \leq 0 \\ e^{-\Delta E/T} & \Delta E > 0 \end{cases}$$

(minimisation; flip sign of ΔE for maximisation). Draw $u \sim U(0,1)$, accept if $u \leq P$.

High T : explore (accept many worse moves). Low T : exploit ($\rightarrow$ hill climbing).

Cooling: geometric $T \leftarrow \alpha T$ ($0 < \alpha < 1$), linear, or logarithmic. Reheating escapes stagnation.

Analogy: cooling metal to a low-energy crystal. Markov-chain convergence to global opt with slow enough schedule.

SA Acceptance Example

$e^{-0.2} \approx 0.818$.

For $\Delta E = 2$: $T=5$: $e^{-0.4} = 0.67$;
 $T=1$: $e^{-2} = 0.135$; $T=0.1$: $e^{-20} \approx 0$.

Lower $T \Rightarrow$ worse moves rarely accepted.

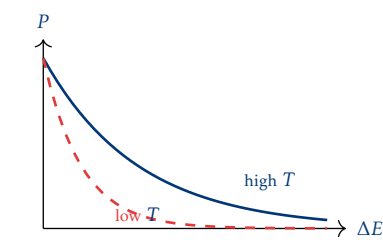
Accept/reject (min),

$E(s)=10$, $E(s')=13$, $\Delta E=3$:

$T=10$: $P = e^{-0.3} = 0.74$, draw $u=0.5 \leq 0.74 \Rightarrow$ accept.

$T=1$: $P = e^{-3} = 0.05$, draw $u=0.5 > 0.05 \Rightarrow$ reject.

SA Acceptance Curve



$P = e^{-\Delta E/T}$: higher T flattens the curve (more uphill moves accepted).

Tabu Search (TS)

Deterministic; escapes minima via memory not randomness.

Candidate set $C(s) = (N(s) \setminus T) \cup A(s)$: non-tabu neighbours plus aspirated moves.

Each step: evaluate $N(s)$, pick best admissible move (even if worse), update tabu list + best-so-far. Tabu (short-term/recency): forbids recently used move attributes for a tabu tenure (e.g. swap matrix $\text{Tabu}(i,j) \leftarrow T$, decrement each iteration; tabu while > 0).

Aspiration: override tabu if move beats best-so-far $E(s') < E(s^*)$ (permission, not forced selection). Long-term/frequency memory: penalise often-used attributes

$\text{Score} = \text{Value} + F(i,j) \Rightarrow$ diversification. Recency-based intensification fixes persistent components.

Tabu prevents cycling, not progress. Tenure too short $\Rightarrow$ cycles; too long $\Rightarrow$ over-restriction (adapt between $L_{\min}, L_{\max}$).

TS Tiny-TSP Example (2-opt)

$s_0 = A-B-D-C-E-A$, $E=25$. Move: cut $(B,D), (C,E)$, add $(B,C), (D,E)$:

$E(s_1) = 25 - (6+4) + (2+2) = 19$; Tabu $\leftarrow \{(B,C), (D,E)\}$, tenure $L=2$.

Next iter: reverse move removes tabu edges $\Rightarrow$ blocked; best admissible $\rightarrow E = 23$ (worse, but avoids cycling).

TS Aspiration Example

$E(s_k)=20$, best $E(s^*)=19$. Candidates: $m_1(\text{tabu}, 17)$, $m_3(18)$, $m_4(21)$.

m_1 is tabu but $17 < 19 \Rightarrow$ aspiration accepts m_1 .

Trajectory Comparison

Method	Uphill?	Memory
Hill Climb	no	none
Sim. Anneal	prob.	none
Tabu	strategic	short/long

6 Genetic Algorithms (Population)

Population vs Trajectory

Population: many candidates

$P(t) = \{x_1, \dots, x_N\}$; built-in parallelism + diversity; escapes local optima more easily.

No Free Lunch: averaged over all problems no search beats random; gains come from injected "active information" via representation, operators, selection, fitness.

Families: GA, Evolution Strategies, Evolutionary Programming, PSO, ACO.

Biology Mapping

Biology	GA	Optim.
Individual	chromosome	candidate
Gene	variable	parameter
Fitness	fitness fn	objective
Environment	problem	constraints

Genotype: encoded form GA

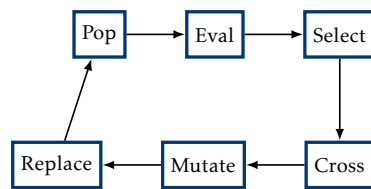
operates on. Phenotype: actual solution. Fitness $f(x)$: scalar quality, only feedback signal.

SGA Cycle (Holland 1975)

Init $P(0) \rightarrow$ evaluate $\rightarrow$ repeat{ select parents $\rightarrow$ crossover $\rightarrow$ mutation $\rightarrow$ replace $\rightarrow$ evaluate }.

GGA: full replacement each generation. SSGA: incremental (few replaced), lower pressure, better diversity.

GA = variation + selection. Selection only $\Rightarrow$ collapse; variation only $\Rightarrow$ random drift.



Representation

Binary $b \in \{0,1\}^L$. Decode integer I to real $x \in [x_{\min}, x_{\max}]$:

$$x = x_{\min} + \frac{I}{2^n - 1} (x_{\max} - x_{\min})$$

Resolution (step): $\Delta x = \frac{x_{\max} - x_{\min}}{2^n - 1}$.

Hamming dist = # differing bits; binary distorts locality:

$7 = 0111$, $8 = 1000 \Rightarrow$ Hamming 4 though values adjacent.

Gray code: consecutive integers differ by 1 bit $\Rightarrow$ smoother landscape.

Real-valued: genotype = phenotype, smooth variation. Permutation: ordering (TSP, scheduling), validity is a hard constraint.

Crossover (rate p_c , 0.6-0.9)

Draw $u \sim U(0,1)$; if $u \leq p_c$ recombine, else copy parents.

1-point: swap tails after cut k .

N-point: alternate segments.

Uniform: per-gene coin flip (max mixing, disruptive).

Real: arithmetic $x^{(c)} = \alpha x^{(1)} + (1 - \alpha)x^{(2)}$; BLX- α samples $[\min - \alpha d, \max + \alpha d]$,

$d = |x^{(1)} - x^{(2)}|$; SBX (index η_c : large=near parents).

Mutation (rate $p_m \approx 1/L$)

Per gene: if $u \leq p_m$ mutate. Bit-flip; swap/scramble/inversion (permutations).

Real: Gaussian

$x'_i = x_i + \epsilon$, $\epsilon \sim N(0, \sigma^2)$; adaptive σ shrinks over generations.

Too low $\Rightarrow$ stagnation; too high $\Rightarrow$ random walk. Maintains diversity / escapes traps.

Selection

FPS / roulette: $p_i = \frac{f_i}{\sum_j f_j}$. Sensitive

to scaling; weak pressure late; shifting all f by a constant changes p_i .

Rank-based: probability by rank (scale/translation invariant).

Tournament: best of k random; pressure set by k ($k=1$ none, large k aggressive); no scaling needed.

SUS: evenly spaced pointers. Elitism: keep best E ; stabilises but raises pressure.

Premature convergence: diversity lost early (high pressure, small N , strong elitism, deceptive landscape).

FPS Example

Selection prob $p_i = f_i / \sum_j f_j$.

Early (spread fitness):

$f = \{40, 20, 10\} \Rightarrow p = \{0.57, 0.29, 0.14\}$

$\Rightarrow$ strong pressure toward best.

Late (similar fitness):

$f = \{101, 100, 99\} \Rightarrow p \approx \{0.34, 0.33, 0.33\}$

$\Rightarrow$ pressure nearly gone (FPS weakness).

Deception (3-bit trap)

Let u = #ones in the block. $f(000) = 4$ (global); $f(u) = u$ for $u \in \{1, 2, 3\}$.

Selection rewards more 1s $\Rightarrow$ population climbs to 111 (local opt); reaching 000 needs a disruptive jump.

Here higher selection pressure speeds up failure (loss of diversity).

Binary GA Example (max x^2 , 5-bit)

Population and fitness $f = x^2$:

$A=01001(9)$, $B=10100(20)$

$C=00110(6)$, $D=11100(28)$

f : 81, 400, 36, 784.

Tournament $k=2 \Rightarrow$ parents $[A, D, B, D]$.

1-point cut after bit 3, $p_c=0.8$; bit-flip $p_m=0.2$; elitism keeps D .

Best improves $28^2 \rightarrow 29^2$; population drifts to larger x .

PMX (permutation crossover)

Copy segment $P_1[c_1:c_2]$ into child; build value mapping from the aligned $P_1 \leftrightarrow P_2$ segment; fill remaining slots from P_2 , repairing duplicates by following the mapping chain until an unused value appears.

Parents (cut after pos 3,6):

$P_1 = 123|456|78$

$P_2 = 572|184|63$

Map $4 \leftrightarrow 1$, $5 \leftrightarrow 8$, $6 \leftrightarrow 4$; child = 87245613.

Swap mutation keeps the permutation valid.

Exploration vs Exploitation

Crossover: large jumps, recombine building blocks (exploration).

Mutation: local refinement + diversity. Selection: exploitation.

Tuning GA = balancing these. GA is anytime; termination (max gens / evals / convergence / target) does not imply optimality. Poor choice when problem is convex/smooth with usable gradients.